Prepatory Mathematics in the Age of Information Chapter One

teilchen010

[2015-10-06 Tue 14:32]

Contents

1 Introduction **1**

2 The so-called real-world **2**

3 Introducing Kathy **4**

4 SICP **5**

5 Algorithms **8**

6 *Pi* from an algorithmic standpoint **9**

7 A Warm-up: Prime Factorization For Unlike Denominators **9**

7.1 A bit of Algebra; Using *symbols* for primes. 10

Abstract

1 Introduction

Once upon a time the main customer of mathematics was the greater world of physics and engineering. While math is just as involved with physics and engineering as ever, we now find it necessary to consider computers and their unique approach to mathematics. Along with classic *continuous* mathematics, computers have ushered in a new era of *discrete* mathematics.

Roughly speaking, *continuous* math is the math of things in motion, e.g., objects and their paths or trajectories, forces measured to the decimal place. In continuous math there is typically a *formula*. On the other hand, *discrete* math deals with separate, *discrete* entities, things are usually reoresented with integers, there are *systems* of finite, non-smoothly associated objects.

Discrete mathematics is largely absent from the American secondary school curriculum, and, except for *linear algebra*, not emphasized in the first undergraduate years of American college either. And yet theoretical computer science is mainly about discrete mathematics. An issue running parallel to the dearth of discrete math is the glaring absence of any cohesive approach to programming computers. Again, secondary education offers very little programming, and outside of computer science programs, college science curricula mostly follow a "catch as catch can" approach to learning to program. For example, when this author took numerical analysis from his math department (1985), the professor offered no advice—let alone training—on how to do the programming assignments. Luckily, some of us stumbled onto FORTRAN and hacked together our assignments, mostly by guesswork . . . and things haven't changed much in the last thirty years.

These deficiencies have been topics of discussion for many years. The first major breakthrough came in the late 1980s from Gerald Sussman and Hal Abelson at MIT with their landmark textbook *Structure and Interpretation of Computer Programs*[1]. Its nickname was *the wizard book* due to the wizardy-looking cartoon character on the cover. SICP served for many years as the text for their MIT Electrical Engineering and Computer Science course *6.001*. SICP was praised by many as a nexus of computer theory and numerical application. However, its use of the Scheme programming language, as well as its legendary "hard math" practice exercises doomed acceptance beyond any but the world's top university computer science programs.

Today the importance of learning math on the computer cannot be emphasized enough, as nearly all real-world mathematics is done on computers. Gone are the days of slide rules, pocket calculators, log and trig tables, gone even are the days of mainframe computers chugging away in some remote clean room at FORTRAN numerical programs. Today's math is done as much on consumer-class computers (like yours and mine) with both free and commercial software, as on massively parallel super computers in clean rooms.

This text aims to be an aid to the instruction of basic mathematics by including the computer and programming at each step. Math will be, first and foremost, in the context of the computer. *If you can code it, you own it* might be our motto. And to *code it* we'll be using the *SICP* standby Scheme, specifically the *Racket* dialect. (See Appendix [] for installation instructions, and Appendix [] for some basic skill-building.)

2 The so-called real-world

In regular math we see *functions*, expressions, equations. A function is a statement, an equation is a statement, a mathematical expression is also a statement of some mathematical relationship, hopefully accurate and true. Math builds,

[1] Harold Abelson and Gerald Sussman, *Structure and Interpretation of Computer Programs; Second Edition* (MIT Press), `http://sarabander.github.io/sicp/html/index.xhtml`

derives, juxtaposes functions, expressions, equations to get at some basic, fundamental truth of the matter at hand. With an equation like $y = y_0 e^{kt}$ we see a factory:

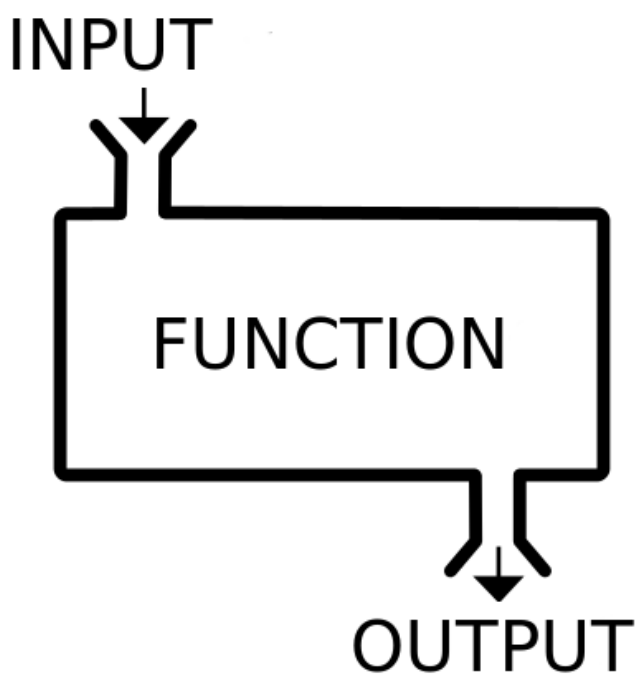


Figure 1: Courtesy of Wikimedia Commons

of sorts that takes a thing y, perhaps a bacteria blob, at an initial starting time $t = 0$, that is, the blob's state at y_0, and multiplies it by Euler's "magic" constant e raised to the power of kt, where t is time and k is a constant, i.e., e^{kt}. What is this for? What does it do? Well, to begin answering this question many mathematics teachers would first want their students to know where the equation came from—maybe not the whole historical rendition of when and who plucked it out of the *mathematical void*—but students should see that it is *derived* using valid, mathematically-legal substitutions and simplifications from a more basic mathematical statement

$$\frac{dy}{dt} = ky$$

. . . then the students do some homework problems, and maybe see in on

a test. And there the ball stops—until a day comes when one of them must use the *exponential rate of growth or decay dependent on initial size* formula in a real-world setting—invariably on a computer; invariably in a much messier situation than the Calculus text problem set.

3 Introducing Kathy

Let us introduce *Kathy*, a new hire at a huge multi-national chemical engineering firm. The last time Kathy saw anything about exponential growth/decay was in her college Calculus course some four years ago. She only remembers plugging numbers into a calculator and writing down the answer on a test. But now her new boss wants her to do some growth factor calculations "*on the computer*". Not sure how to proceed, she asks around discreetly, to which someone says "*Oh, I think you could use* Excel *for that sort of thing.*"

Googling, she finds a page on an Excel exponential growth formula—but it's done in some computer code, and, on top of that, it doesn't look the same as the formula she dimly recalls from college. But she's got a good head for math and a strong desire to prove herself, so she keeps after it until something works and it looks right. She takes the printout to her boss . . . and he stares at it unsmiling, looking almost like he's been insulted. Finally he hands it back to her, saying /"Have Roy set you up a Matlab account."/ Yes, sir. And so Roy, her unit's sysadmin, dutifully installs said software. But just as she begins to ask "*What do I do now?*" he hands her a three pound Matlab manual. "*Study this*," he says, smirking, an eyebrow raised.

The afternoon passes as she wades through the book's intro and first chapter, playing around with the examples on her new Matlab account. But during lunch in the cafeteria, she overhears a group of colleagues talking about *matrices*, then something called *Python*, then something called *Clojure*. She asks them to explain, and they say Matlab is good for some things, but their group is "writing applications." Oh.

Later, as she's at a restaurant with her friends, as they're talking about something on *YouTube*, she sighs and wishes she had seen some of this *numerical* stuff before being thrown into the shark tank. Some sort of bridge would have been nice between the world of school math—with theories and practice problems from textbooks worked out with pencil, paper, and calculator—and the proverbial deep end of the real world where everything seemed to revolve around computers and software. She realizes that she must transform herself into a *numerical analyst* in no time flat.

Later that night, about two chapters into an online *Numerical Analysis* textbook, she surmises the whole point is to find strategies to *translate* the nice, clean math of her textbooks into forms that can be carried out on computers. In effect, numerical methods are formulas to apply formulas. . . .

4 SICP

An online search led her to something called the Structure and Interpretation of Computer Programs, which, as she learned, was both a book and a course at MIT. She eventually found a nice creative commons online version of the text and started reading.

If numerical analysis was about turning math concepts into workable methods, then SICP was about turning those methods into computer code. The authors Sussman and Abelson presented the *functional* programming language *Scheme* as the most natural fit for turning mathematical methods into code. As she Googled *functional programming*, various sources said the functional style was in numerical application preferrable to the *imperative* style, mainly because imperative coding was apt to be too verbose in the translation of task into actual code, while functional coding stayed closer to mathematics by mimicking the actual functions.

One particular sticking point about imperative programming seemed to be how an equation like $x = 2$ was not really an equation in the math sense, but more about storing 2 in a place in computer memory referred to as x. And why is that a problem? If $x = 2$ is not a true mathematical statement, rather a statement about storage, an imperative program might later reassign x to "equal" something else—which, of course, violates the idea of what x really is—at least mathematically-speaking . . . she wasn't really sure why that was such a big deal, but she read on.

The first chapter seemed to be reproducing with Scheme code much of what a basic pocket calculator does. She had briefly seen the programming language *C* in a beginning computer science course, but dropped it (and forgot about a comp-sci minor as well) after getting totally frustrated with how much time and effort it took to do the simplest things. Rules, rules, rules! But what she now saw was nothing like C. Functions were built as *lists* of nested parentheses. Apparently, Scheme was a derivative of Lisp, which is based on *expressions* packaged as lists to be evaluated, rather than *statements* to be executed. . . .

She wasn't sure what all that exactly meant, but in math, you would plug numbers into a formula and get back an answer. For example, the Scheme way of setting up $f(x) = x^2$ was this:

```
(define (mysquare x)
  (* x x))
```

Basically, the expression *let* $f(x) = x^2$ is `(define f x (* x x))` but with Scheme, you're encouraged to give your functions more creative names, like `myspace` rather than f. Now, if we plug in 2, `(mysquare 2)`, we get 4. Now, contrast this Scheme code with C code:

```
#include <math.h>   /* for sqrt() function */
#include <stdio.h>
int main( )
{
```

```
  /* variable named x with floating-point data type */
  float x;
  /* standard output  writing a positive floating number */
  printf("\nEnter one positive number (1, 2, 3. . .): ");
  /* standard input reading floating number */
  scanf("%f", &x);
  printf("\nx = %f ", x);
  printf("\nSquare for x is = %f", x * x);
  return 0;
}
```

To be fair, Scheme is normally an *interpreted language*, i.e., it has a running *REPL* session, where users simply feed it code live and get results back immediately, while C code must be compiled, then executed in order to interact with it. But Kathy had a bad flashback seeing all that code just to get something as simple as a square-a-number function done.

A bit odd, though, was Scheme's *prefix* notation, i.e., how the code wrote *x times x* as `(* x x)`, but the examples in SICP of `(+ 1 3 5 7 11)` made the light go on in her head how prefix notation was really handy *and* followed the Scheme philosophy that the first thing in any list is usually a function—even if it's just a basic operator like plus (+). In fact, she found she could write her own plus if she wanted:

```
(define myplus +)
```

. . . and it works just like normal `+`. And even cooler was not just giving `+` a alias/pseudonym, but a real definition:

```
(define (my+func . numbers)
  (if (null? numbers)
      0
      (+ (car numbers) (apply my+func (cdr numbers)))))
```

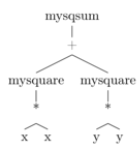
Kathy had found this code online, and wasn't sure what it was doing, but she ran it in the REPL and it worked: `(my+func 1 2 3 4 5)`, which gave the correct result of `15`.

Reinventing the *addition unary operator* wasn't much to write home about, but it drove home the idea that addition can also be thought of as a function, just like any $f(x)...$ construct, again, with longer, more expressive names than just f or g or h as she knew from math.

But then what about all those parentheses? Actually, they seemed to be used pretty much like math used them, i.e., to bracket off things meant to be together. In fact, they were supposed to be *lists*, specifically, nested lists. And so everything is like a tree. She looked at the SICP "sum of squares" function, which took two squares and added them together:

```
(define (mysqsum x y)
  (+ (mysquare x) (mysquare y)))
```

Then she drew a picture of `mysqsum` as a tree.



Next in SICP she saw this:

$$|x| = \begin{cases} |x & \text{if} \quad x \geq 0, \\ -x & \text{if} \quad x < 0. \end{cases}$$

which was then translated into this Scheme code:

```
(define (myabs x)
  (cond ((>= x 0) x)
        ((< x 0) (- x))))
```

It gives the *absolute value* of any number plugged in, e.g., `(myabs -1)` gives 1, `(myabs 0)` gives 0, and `(myabs 1)` gives 1.

Puzzling out the code, she saw that `cond` means *condition*, and that the parens are there to section off each condition to consider. First consider whether x is greater than or equal to 0 with `((>= x ) x)`. The `(>= x)` part is called a *predicate*, a form that resolves to either true (`#t` in Scheme) or false (`#f` in Scheme). If `#t`, `myabs` just give back the `x` you plugged in and skips the other `cond` form. Otherwise, go on to the next form, `((< x 0) (- x))`, which gives back "the opposite" of the negative `x`, a positive `x`. It took her a second or two to grok what they were saying in SICP about how everything inside the deepest nested parens must be evaluated before considering what's next; e.g., with `((>= x 0) x)`, Scheme first has to do the inside form, `(>= x 0)`. Once it gets a true or false to the predicate `>=`, it acts accordingly. So, e.g., if `x` is 1 or 0, the `(>= x 0)` part (form) evaluates to `#t` and `x` as you fed it into `(myabs 1)` wins—and then the next chunk, `((< x 0) (- x))` is skipped. That's how a `cond` works: It tests conditions, as many as you like! You can stack up a hundred conditions to test for, and as soon as one is true (`#t`), Scheme skips to the next section outside the `cond` closing parens.

Okay, so Kathy wasn't opposed to learning a computer language after all. Sure, it had some basic rules, *syntax* rules, as they're called, but it all seemed doable with Scheme. And not some death march through the weeds and brambles like C was.

5 Algorithms

That weekend Kathy phoned her older brother, Lee, who had recently moved to Sweden to work for a big telecom company. As she told him about her new job, Lee kept chuckling in a knowing way. He knew exactly where she was at, having had to retool himself from an electrical engineer into a "numerical specialist," too. Now he was doing computer simulations, writing code to control telephone systems. He agreed that functional programming was the way to go; he and his people used a language called *Erlang*, which was good at coordinating distributed computing. He also gave her some tips about the scientific computing world. Yes, he had also looked into Scheme and told her about a good, free entry-level book called *Simply Scheme* by Cal Berkeley computer science professor Brian Harvey.

Lee also said that although computers and computer science are quite mathematical, the world of computers and math aren't perfect partners. He told the Edsger Dijkstra quote about how computer science is no more about computers than astronomy is about telescopes. Lee noted that the main use for computers was *data management*, and that required efficient *algorithms* to process the data.

Later, Kathy found the word algorithm popping up repeatedly. She watched a Khan Academy video on algorithms. It said . . . *an algorithm is a set of steps to accomplish as task.* That reminded her of something she saw in a book she

had just bought at a used book store, *Electronic and Computer Math*[2]. It often used an *algorithmic* approach to laying out a math problem. For example, there was a section on adding a subtracting fractions with unlike denominators. She remembered from way back in junior high learning to do *prime factorization*, which was a trick to find the *lowest common multiple*, which was, in turn, the *common denominator*. Like an algorithm, the book layed out steps to getting an answer.

- find the common denominator (CD)
- change the denominator of all fractions to this CD
- divide the CD by the old denominator of each fraction
- multiply this number by the old numerator to get the new numerator
- add (or subtract) the fractions, combining all the like terms in the numerator.

Obviously, this sort of algorithmic breakdown was necessary to tell a computer how to do unlike fraction addition. But then a fraction was just another name for a

6 *Pi* from an algorithmic standpoint

. . .for example, Kathy found out *pi*, the ratio of a circle's circumference and its diameter, can be derived in many ways. And even before the dawn of computers, numerical methods were used to get at a real solution to the basic $\pi = {}^{C}/_{d}$.

7 A Warm-up: Prime Factorization For Unlike Denominators

Seriously large denominators require the *prime factoring* method in order to get like-denominators.

Prime factorization means *factoring out* the *prime numbers* of a number. For example:

$18 = 2(9)$ or 18 equals 2 times 9 (using Algebra notation for *times*)

but 9 is not a prime number. So we must *factor* it:

$9 = 3(3)$

Now we can say $18 = (2)(3)(3)$ (again, using Algebra notation for *times*) or, using *powers*, $18 = 2(3^2)$ where 3^2 is the same as $3 \cdot 3$ or 3 times 3.

Before going any further, we should maybe talk about *prime numbers* or just *primes*. A prime is simply a number that has no possible factorization other than 1 and itself. So this table:

[2] Bill Deem and Tony Zannini, *Electronics and Computer Math; Seventh Edition* (Prentice Hall), 124.

2	3	5	7	11	13	17	19	23	29	31	37	41	43	47	53	59	61	67	71	73	79	83

shows the first 29 primes. Again, nobody in this table is divisible by anything other than 1 and that number itself; no other numbers will divide into a prime number.

When you think of numbers in terms of primes, any number is just a bunch of primes multiplied together—sometimes repeatedly, e.g.,

$$\begin{aligned} 38 &= (2)(18) \\ &= (2)(2)(9) \\ &= (2)(2)(3)(3) \end{aligned}$$

or

$$38 = (2^2)(3^2)$$

So the prime factors of 38 are just the primes 2 and 3. We can now say the *factors of any number are just prime numbers*. Take a look at this link for a tables of the first $1,000$ numbers as factors of primes.

7.1 A bit of Algebra; Using *symbols* for primes.

- If we match the primes to letters, like `a, b, c`,

a	b	c	d	e	f	g	h	i
2	3	5	7	11	13	17	19	23

  we can write the denominators of fractions like this:

$$\frac{1}{85} \equiv \frac{1}{(5)(17)} \equiv \frac{1}{cg}$$

  where cg is (as above) c times g. ($\equiv$ means equivalent or "the same as") And for another fraction

$$\frac{1}{276} \equiv \frac{1}{(2^2)(3)(23)} \equiv \frac{1}{a^2bi}$$

  As you can see, the *factors* in the denominator of both fractions are made up entirely of primes. Now we can pick a combination of these primes to make a *lowest common denominator*, which is a fancy way of saying the smallest denominator that both original denominators are some multiple of.

$$\frac{1}{85} + \frac{1}{276} \quad \text{or} \quad \frac{1}{cg} + \frac{1}{a^2bi}$$

Broken down into their prime factors, we see that the two fractions have unique prime factors, i.e., they do not share any prime factors. This means a common denominator between the two must include *all* the prime factors *and* their powers:

$$\frac{1}{cg} + \frac{1}{a^2bi} \equiv \frac{(1)(a^2bi) + (1)(cg)}{(cg)(a^2bi)}$$

If we substitute the original numbers back in. . .

$$\frac{(1)(2^2)(3)(23) + (1)(5)(17)}{(5)(17)(2^2)(3)(23)}$$

By the way, this is the extreme condition where the common denominator cannot be anything less than simply multiplying the original denominators. Why? What happened? The two denominators—when factored down to their respective prime factors—had *nothing*, no primes, in commmon.

But what do we do with the numerators? We're not allowed to change the value of the fractions in our attempt to add them—and we don't. And so we use an Algebra trick to keep the fractions the same value. The trick, of course, is multiplying something by 1, which, of course, doesn't change it:

$$\left(\frac{1}{85}\right)\left(\frac{276}{276}\right) + \left(\frac{1}{276}\right)\left(\frac{85}{85}\right) \quad \text{or} \quad \left(\frac{1 \cdot 276}{85 \cdot 276}\right) + \left(\frac{1 \cdot 85}{276 \cdot 85}\right)$$

then

$$\frac{(1)(276) + (1)(85)}{(85)(276)} = \frac{276 + 85}{23460} = \frac{361}{23460}$$

Let's look at something where the denominators might be large, but have more primes in common. How about

$$\frac{1}{40} + \frac{1}{81} + \frac{1}{92} \quad \text{or} \quad \frac{1}{a^3c} + \frac{1}{b^4} + \frac{1}{a^2i} \quad \text{or} \quad \frac{1}{(2^3)(5)} + \frac{1}{3^4} + \frac{1}{(2^2)(23)}$$

here we see the primes 2, 3, 5, and 23 in use. Now we need to establish what the *lowest common denominator* (LCD) should be. The old *brute force* method of finding an LCD would be to simply multiply all the denominators together $(40 \cdot 81 \cdot 92)$ and then multiply the numerators by

whatever was need to keep it honest. We, however, don't need such *mathématiques brutale*. We're clever enough to sense that if a prime in one denominator is also found in another denominator, *we can simply disregard the lesser power of that prime and use the greater power to build our LCD.* Our example has just such a situation. Notice above that we have the prime 2 in both $\frac{1}{40}$ and in $\frac{1}{92}$. Notice also that $\frac{1}{40}$ has 2^3 while $\frac{1}{92}$ has just 2^2. The beauty of the **prime factorization** method is that we can throw out the lesser power (here 2^2) and build our LCD out of the greater power 2^3

$$\frac{1}{(2^3)(5)} + \frac{1}{3^4} + \frac{1}{\cancel{(2^2)}(23)}$$

so now our denominator will be made up what we might call the *significant* or *principle* primes: $(2^3)(3^4)(5)(23)$ or $74,520$. Still *ugly*-big, but smaller by the factor of 2^2 we were able to drop, i.e., $74,520 < 298,080$ by a *factor* (note the common English use of *factor*) of 4. Now, getting the numerators right will simply be the Algebra trick of multiplying each fraction by a version of 1, i.e., not changing its actual value, but making each fraction have like-denominators so we may add them. Again, in order to transform $\frac{1}{40}$ into a version with our LCD $74,520$, then $\frac{1}{81}$, lastly, $\frac{1}{92}$, we multiply each by a different version of 1, in the form of $\frac{a}{a}$ where a is that number, which, when multiplied by the fraction's denominator, will give the LCD, in this case $74,520$:

$$\left(\frac{(3^4)(23)}{(3^4)(23)}\right)\left(\frac{1}{40}\right) + \left(\frac{(2^3)(5)(23)}{(2^3)(5)(23)}\right)\left(\frac{1}{81}\right) + \left(\frac{(2)(3^4)(5)}{(2)(3^4)(5)}\right)\left(\frac{1}{92}\right)$$

or

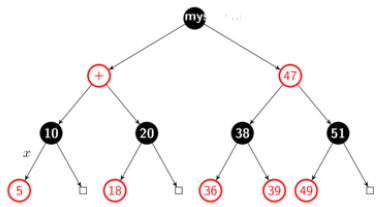
$$\left(\frac{1863}{1863}\right)\left(\frac{1}{40}\right) + \left(\frac{920}{920}\right)\left(\frac{1}{81}\right) + \left(\frac{810}{810}\right)\left(\frac{1}{92}\right)$$

or

$$\frac{1863}{74520} + \frac{920}{74520} + \frac{810}{74520} = \frac{3593}{74520}$$

Note, we multiply each fraction by the factors the fraction's denominator doesn't contain. So with $\frac{1}{92}$, which contributed 2^2 and 23 to the LCD, we make the $\frac{\textit{something}}{\textit{something}}$

Another way to figure out the numerators—or what we should multiply each fraction's numerator by—is to simply divide the LCD by the original denominator, e.g., for $\frac{1}{40}$ we have $74,520 \div 40 = 1,863$, and yes, $1,863$ is the same answer as before, i.e., we multiply $\frac{1}{40}$ by $\frac{1863}{1863}$ to get $\frac{1}{40}$ represented in terms of our LCD.


my
47
10
20
38
51
x
5
18
36
39
49